Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	4 (Term 1)	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will develop interactive programs that collect information and present clear output. They will explain the purpose of prompts and recognise that input() returns a string. Students will convert numeric input when calculation is required and use a supplied decision structure to check a permitted range. They will distinguish validation from verification and recognise that conversion alone does not make data valid.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, discussion of worked examples, the completed student tasks and the final checks in both lessons. Check prompts, stored input and a correct summary for two trials. Check 0, 100 and an out-of-range number; explain why valid data can still be inaccurate. Check that students can explain their reasoning and apply the skill without relying on the worked example.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	14 September 2026	By the end of this lesson, students will be able to collect input and produce meaningful output in Python.	**H:** Ask why a program needs a prompt before accepting input. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain input(), print() and that input() returns a string. Discuss the purpose of an informative prompt and explain how the returned string is stored for later use. Distinguish input from output and avoid collecting real personal information during classroom practice. **WD:** Build an interactive name greeting with clear prompts and output. Ask for a fictional club member and favourite activity. Store the responses separately and print a readable confirmation. Run the program with	**Support:** Provide a prompt-variable-output planning sheet and a working single-input example. Colour-code the stored response and its later use. Students add the second input after explaining the first example. **Extension:** Students improve the layout of the confirmation without duplicating input statements. They explain why an empty response is still a string and suggest a requirement for handling it.

			different responses and ask which parts of the output change. **YD:** Create a short club-registration dialogue using fictional names and display a readable summary. Students include two clear prompts, meaningful variable names and a complete confirmation message. They test two fictional registrations and ask a partner whether the prompts make the expected response clear. **C:** Check prompts, stored input and a correct summary for two trials. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
2	18 Septemb er 2026	By the end of this lesson, students will be able to convert numeric input and check a simple permitted range.	**H:** Predict what happens when two input strings are added. Invite students to explain their prediction before demonstrating the example. Use their responses to identify the idea that needs clarification. **ID:** Explain int(), float() and range checks; distinguish validation from verifying accuracy. Explain why converting a numeric string permits arithmetic, while a range check decides whether the resulting value is permitted. Validating data does not prove that the user entered the factually correct value. **WD:** Convert a supplied numeric score and check whether it lies between 0 and 100. Use a score of 50 to demonstrate int() and compare the result with the original string. Show a supplied range-checking structure, then	**Support:** Provide a working range-check template because independent selection is developed later. Model substituting the score into each comparison and use a number line to show the inclusive valid interval. **Extension:** Students compare int() and float() for a decimal score and explain which matches the stated requirement. They identify the risk of non-numeric input and describe an appropriate error message in plain language.

			trace 0, 100 and 101 through it. Discuss non-numeric input as a limitation to address later. **YD:** Build a score checker using numeric test inputs; reject values outside 0–100. Record expected results first. Students use the supplied decision structure to complete the checker and record expected and actual results. Include both valid endpoints and at least one invalid numeric value; explain one difference between conversion and validation. **C:** Check 0, 100 and an out-of-range number; explain why valid data can still be inaccurate. Ask students to support their answer with a specific example from their work, and record any misconception that needs a further demonstration.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Pearson Edexcel International GCSE Computer Science Student Book: Chapters 5 and 9
- Teacher computer and classroom display
- Classroom computers with Python 3 editor and individual save folders
- Prepared algorithm, tracing or Python practice tasks aligned with the lesson objectives
- Whiteboard and markers
- Student notebooks and pencils
- Success criteria, test tables and teacher observation checklist



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?